

SQL INTERVIEW PLAYBOOK

Data Science • Data Analyst • Analytics • ML Interviews

Basics → Advanced SQL → Real Interview Problems

A practical revision guide with interview-ready answers, query patterns, and data-science examples.

#	Section	Focus
01	SQL Fundamentals	Core concepts, keys, NULL, DDL/DML
02	Filtering & Aggregation	WHERE, GROUP BY, HAVING, aggregates
03	Joins	INNER, LEFT, RIGHT, CROSS, SELF
04	Subqueries & CTEs	Nested and multi-step analysis
05	Window Functions	Ranking, LAG, LEAD, running totals
06	Advanced SQL	CASE, COALESCE, indexes, views, transactions
07	SQL for Data Science	Cleaning, features, KPIs, business analysis
08	Query Challenges	Common coding-round problems
09	Rapid-Fire Viva	One-line interview answers

01 • SQL FUNDAMENTALS

Q1. What is SQL?

Answer: SQL (Structured Query Language) is used to store, retrieve, manipulate, and analyze data in relational databases.

Q2. What is a DBMS?

Answer: A Database Management System is software used to create, store, manage, and access databases.

Q3. What is a relational database?

Answer: A database that stores data in related tables consisting of rows and columns.

Q4. What is a primary key?

Answer: A column or combination of columns that uniquely identifies each row and cannot contain NULL.

Q5. What is a foreign key?

Answer: A column that references a key in another table and maintains relationships between tables.

Q6. What is a candidate key?

Answer: A column or set of columns that can uniquely identify rows and can therefore qualify as a primary key.

Q7. What is a composite key?

Answer: A key made from two or more columns used together to uniquely identify a row.

Q8. What is NULL?

Answer: NULL represents missing or unknown data. It is not equal to 0 or an empty string.

Q9. How do you check NULL?

Answer: Use IS NULL or IS NOT NULL; do not use = NULL.

Q10. DDL vs DML?

Answer: DDL defines database objects, such as CREATE and ALTER. DML changes data, such as INSERT, UPDATE, and DELETE.

Q11. DELETE vs TRUNCATE vs DROP?

Answer: DELETE removes rows and can use WHERE; TRUNCATE removes all rows; DROP removes the object and its structure.

02 • FILTERING & AGGREGATION

Q1. What does DISTINCT do?

Answer: It removes duplicate result combinations.

Q2. What are aggregate functions?

Answer: COUNT, SUM, AVG, MIN, and MAX summarize multiple rows.

Q3. COUNT(*) vs COUNT(column)?

Answer: COUNT(*) counts rows; COUNT(column) counts non-NULL values.

Q4. What does WHERE do?

Answer: WHERE filters individual rows before grouping.

Q5. What does GROUP BY do?

Answer: GROUP BY creates groups so aggregate calculations can be performed per group.

Q6. What does HAVING do?

Answer: HAVING filters groups after GROUP BY and aggregation.

Q7. WHERE vs HAVING?

Answer: WHERE filters rows; HAVING filters grouped results.

Q8. What is CASE?

Answer: CASE adds conditional logic to a SQL query and is useful for labels and feature creation.

Q9. What is COALESCE?

Answer: COALESCE returns the first non-NULL expression.

03 • JOINS

Q1. What is a JOIN?

Answer: A JOIN combines related rows from multiple tables using a join condition.

Q2. INNER JOIN?

Answer: Returns only rows with matches in both tables.

Q3. LEFT JOIN?

Answer: Returns every row from the left table and matching rows from the right table.

Q4. RIGHT JOIN?

Answer: Returns every row from the right table and matching rows from the left table.

Q5. CROSS JOIN?

Answer: Returns every possible combination of rows from both tables.

Q6. SELF JOIN?

Answer: A table is joined with itself, useful for hierarchical relationships such as employee-manager.

Q7. How do you find customers with no orders?

Answer: LEFT JOIN customers to orders and filter where the order key IS NULL.

04 • SUBQUERIES & CTEs

Q1. What is a subquery?

Answer: A query inside another SQL statement.

Q2. What is a CTE?

Answer: A Common Table Expression uses WITH to create a named temporary result for a query.

Q3. Why use CTEs?

Answer: They make complex, multi-step queries easier to read, debug, and explain.

Q4. What is a correlated subquery?

Answer: A subquery that references a column from the outer query.

Q5. CTE vs subquery?

Answer: Both can structure intermediate results; CTEs are often clearer for multi-step logic and can be referenced more cleanly within the statement.

05 • WINDOW FUNCTIONS

Q1. What is a window function?

Answer: It performs calculations across related rows without collapsing them into one row per group.

Q2. ROW_NUMBER vs RANK?

Answer: ROW_NUMBER gives each row a unique sequence; RANK gives ties the same rank and leaves gaps.

Q3. RANK vs DENSE_RANK?

Answer: RANK leaves gaps after ties; DENSE_RANK does not.

Q4. What is PARTITION BY?

Answer: It divides rows into independent groups for a window calculation.

Q5. What does ORDER BY do inside OVER()?

Answer: It defines the order in which the window calculation is evaluated.

Q6. What is LAG()?

Answer: It retrieves a value from a previous row.

Q7. What is LEAD()?

Answer: It retrieves a value from a following row.

Q8. How do you calculate a running total?

Answer: Use SUM(value) OVER (ORDER BY date).

06 • ADVANCED SQL

Q1. What is an index?

Answer: A structure that can speed up data retrieval but adds storage and can increase write overhead.

Q2. What is a view?

Answer: A saved SQL query that behaves like a virtual table.

Q3. What is a stored procedure?

Answer: A named, executable database program that can contain SQL logic and parameters.

Q4. What is a trigger?

Answer: A database object that automatically executes when defined events occur, such as INSERT or UPDATE.

Q5. What is normalization?

Answer: Organizing data to reduce redundancy and improve data integrity.

Q6. What is a transaction?

Answer: A logical unit of database work that can be committed or rolled back.

Q7. What is ACID?

Answer: Atomicity, Consistency, Isolation, and Durability—the key properties of reliable transactions.

07 • SQL FOR DATA SCIENCE

These questions connect SQL syntax with the real work of preparing and analyzing datasets.

Q1. How is SQL used in data science?

Answer: SQL is used to extract data, combine sources, clean records, calculate KPIs, create analytical datasets, and engineer features.

Q2. How do you identify duplicates?

Answer: Use GROUP BY on the identifying columns and HAVING COUNT(*) > 1. For record-level deduplication, ROW_NUMBER() is often useful.

Q3. How do you remove duplicates while keeping the latest record?

Answer: Partition by the business key, order by the timestamp descending, assign ROW_NUMBER(), and keep rn = 1.

Q4. How do you find the second-highest salary?

Answer: Use DENSE_RANK() or a subquery. DENSE_RANK is safer when duplicate salaries exist.

Q5. How do you find the top 3 salaries per department?

Answer: Use DENSE_RANK() OVER(PARTITION BY department ORDER BY salary DESC) and filter rank <= 3.

Q6. How do you calculate monthly revenue?

Answer: Group sales by month and calculate SUM(revenue).

Q7. How do you calculate month-over-month growth?

Answer: Use LAG() to get the previous month's value and compare current vs previous.

Q8. How do you create a feature in SQL?

Answer: Use CASE WHEN to transform business rules into numeric or categorical features.

Q9. How do you handle missing values?

Answer: Understand the reason for missingness, then use filtering, CASE, COALESCE, or a justified imputation rule.

Q10. How do you validate a dataset after a JOIN?

Answer: Check row counts, duplicate keys, NULL rates, expected totals, and whether the join cardinality matches the business relationship.

Q11. What is data leakage in SQL feature engineering?

Answer: It occurs when a feature uses information that would not have been available at prediction time.

Q12. How can SQL support exploratory data analysis?

Answer: Use distributions, counts, aggregates, segmentation, time trends, missing-value checks, and outlier investigation.

■ **Interview Tip:** Always explain assumptions, NULL handling, duplicate handling, and what the result means for the business—not only the syntax.

08 • MUST-KNOW QUERY CHALLENGES

1. Second-highest salary

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

2. Duplicate emails

```
SELECT email, COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

3. Top 3 salaries per department

```
SELECT *
FROM (
```

```

SELECT e.*,
       DENSE_RANK() OVER (
         PARTITION BY department_id
         ORDER BY salary DESC
       ) AS rnk
FROM employees e
) x
WHERE rnk <= 3;

```

4. Latest order for each customer

```

SELECT *
FROM (
  SELECT o.*,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date DESC
         ) AS rn
  FROM orders o
) x
WHERE rn = 1;

```

5. Customers with no orders

```

SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;

```

6. Running revenue

```

SELECT order_date,
       revenue,
       SUM(revenue) OVER (ORDER BY order_date) AS running_revenue
FROM sales;

```

7. Department average salary

```

SELECT department_id,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;

```

8. Employees earning above department average

```

SELECT e.*
FROM employees e
JOIN (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department_id
) a
ON e.department_id = a.department_id
WHERE e.salary > a.avg_salary;

```

09 • RAPID-FIRE VIVA

Topic	Interview answer
SQL	Language used to work with relational data.
Primary Key	Uniquely identifies a row.
Foreign Key	References a key in another table.
WHERE	Filters rows.
HAVING	Filters groups.
GROUP BY	Creates groups for aggregation.
JOIN	Combines related rows.
CTE	Named temporary result created using WITH.
Window Function	Calculates across rows without collapsing them.
RANK	Ranks with gaps after ties.
DENSE_RANK	Ranks without gaps after ties.
ROW_NUMBER	Assigns a unique sequence number.
LAG	Gets a previous row's value.
LEAD	Gets a next row's value.
COALESCE	Returns the first non-NULL expression.
CASE	Conditional SQL expression.
INDEX	Can improve lookup performance.
VIEW	Saved query represented as a virtual table.
TRIGGER	Automatically runs for defined database events.
PROCEDURE	Stored executable SQL program.
NORMALIZATION	Reduces redundancy and improves data integrity.
TRANSACTION	A logical unit of database work.

■ FINAL INTERVIEW FORMULA

Understand → Write → Explain → Validate → Optimize.

Clarify the requirement, write the simplest correct query, explain the logic, check NULLs and duplicates, then discuss performance when relevant.